

轻量级 CCI/DDI 网络 + Laplacian/JointNMF + SR-OT 方案 v2

补充：耗时估算、现有代码关系、Laplacian/JointNMF 原理、OT 改进路线

根据当前代码中 `clean_expression_cci_mix`、`laplacian`、`joint_nmf`、`sr_ot`、`_ot_common` 的结构整理

1 先回答几个关键问题

1.1 问题 1：能不能先测输入大小，估算大概要跑多久？

可以，而且应该做。建议加一个 **profile-only / preflight** 流程：只读每个 organ、time、layer 的节点数、边数、LR 文件数和 PIJ source-target 规模，不真正构造完整特征、不真正跑 Sinkhorn。这样可以先得到每个 kernel 的大小和内存估算。

当前代码里已经有一部分基础：`run_mignet_vertical.py` 有 `--progress` 和 `--progress-log`；`_ot_common.py` 在每个 PIJ kernel 开始时会输出 `shape=[n_source, n_target]` 和 `estimated_bytes`。但 `run_mignet_vertical_ablation.py` 暂时没有把 `progress` 参数暴露出来，建议补上。

定义一次 PIJ 任务中 source 节点数为 n_s ，target 节点数为 n_t ，保留的结构特征维度为 d ，Sinkhorn 迭代次数为 I 。这里， n_s 表示时间点 t_0 的节点数量； n_t 表示时间点 t_1 的节点数量； d 表示低维结构特征维度； I 表示 OT 求解的最大迭代次数。

现有 dense OT 的核心压力来自成本矩阵和 kernel 矩阵，规模近似为：

$$N_{\text{entry}} = n_s n_t. \quad (1)$$

这里， N_{entry} 表示一个 source-target 成本矩阵中的元素个数。

CPU float64 模式下，当前代码估算约为：

$$B_{\text{cpu}} \approx 6 \times 8 \times n_s n_t \text{ bytes} = 48 n_s n_t \text{ bytes}. \quad (2)$$

这里， B_{cpu} 表示 CPU 内存估算；8 表示 float64 每个元素 8 字节；系数 6 表示成本矩阵、kernel、中间归一化结果等多个 dense 数组的安全倍数。

GPU float32 模式下，当前代码估算约为：

$$B_{\text{gpu}} \approx 5 \times 4 \times n_s n_t \text{ bytes} = 20 n_s n_t \text{ bytes}. \quad (3)$$

这里， B_{gpu} 表示 GPU 显存估算；4 表示 float32 每个元素 4 字节；系数 5 是 CUDA backend 中的安全倍数。

时间复杂度可以粗略写成：

$$T_{\text{OT}} \propto I \cdot n_s n_t. \quad (4)$$

这里， T_{OT} 表示 OT 阶段耗时。这个公式的意思是：即使特征已经降维，只要 PIJ 仍然是 dense source-target 矩阵，节点数乘积仍然是主要瓶颈。

建议实际使用两层估算：

1. **静态估算**: 先输出每个 organ/layer/time 的 n 、边数 $|E|$ 、LR 文件数 $|Q|$ 、PIJ 矩阵 $n_s \times n_t$ 、估计内存 B 。
2. **小样本校准**: 只跑 heart、一个 level pair、一个 time pair, 例如 11.5 到 12.5, 记录真实 elapsed seconds, 再按 $\sum n_s n_t$ 比例外推全量耗时。

如果第一个 pilot 的 PIJ 耗时是 $\hat{T}$, pilot 的矩阵元素数是 $\hat{N} = n_s n_t$, 全量任务集合是 $\mathcal{R}$, 其中第 r 个任务的矩阵元素数是 N_r , 则外推为:

$$T_{\text{full}} \approx \hat{T} \cdot \frac{\sum_{r \in \mathcal{R}} N_r}{\hat{N}}. \quad (5)$$

这里, T_{full} 表示全量 PIJ 阶段预计耗时; $\mathcal{R}$ 表示要跑的所有 lower/upper kernel 任务集合; N_r 表示第 r 个任务的 source-target 元素数。

1.2 问题 2: 报告里的 Laplacian / JointNMF 和代码里已有的是一样的吗?

不完全一样。名字上对应已有方法, 但我报告里建议的是组合新方法。

方法	当前代码已有	报告建议新增
laplacian	已有。它从 lower_graphs/upper_graphs 计算普通 Laplacian eigenvector embedding, 然后直接用于指标或 cosine kernel。	laplacian_sr_ot。先用 Laplacian-HKS 或 sign-anchored Laplacian 得到结构特征, 再和 SR 成本一起送入 OT。
joint_nmf	已有。它对 context.lower_mats 和 context.upper_mats 分别做 temporal joint NMF。若网络是 clean_expression_cci_mix, 输入仍是大矩阵。	joint_nmf_sr_ot。先把输入矩阵换成 compact CCI/DDI in-out 特征, 再做更安全的 source-only dictionary NMF, 最后和 SR 成本一起送入 OT。
sr_ot	已有。它读取 sr 或 potency_score, 构造标量差异成本, 再跑 Sinkhorn OT。	不单独使用纯 SR, 而是作为结构 OT 的一个约束项: structure + SR。

也就是说, 现有代码能给我们复用框架, 但还没有你这次想要的“**Laplacian/JointNMF + SR 共同产生 PIJ**”的方法。

1.3 问题 3: Laplacian 为什么能让计算量下来? 它是不是降维?

是的, 它本质上可以理解为一种**图结构降维/图谱表征**。当前重的地方是: 每个节点有一个很长的表达向量和 LR/CCI 特征向量。假设原始特征维度是 p , 节点数是 n , 原始矩阵大小是 $n \times p$ 。这里, p 表示表达基因数加上 LR/CCI 特征数。

Laplacian 不从 $n \times p$ 的大矩阵出发, 而是从稀疏图邻接矩阵 A 出发。设图边数为 $|E|$, 只保留 K 个谱分量, 最终每个节点得到 K 维或 H 维特征。这里, A 表示 CCI/DDI 加权邻接矩阵; $|E|$ 表示非零边数量; K 表示保留的特征向量数量; H 表示 HKS diffusion scale 数量。

普通 Laplacian embedding 为:

$$L\phi_m = \lambda_m \phi_m, \quad m = 1, 2, \dots, K. \quad (6)$$

这里, L 表示图拉普拉斯矩阵; ϕ_m 表示第 m 个特征向量; λ_m 表示对应特征值。

如果用 HKS, 节点 i 的第 h 个尺度特征是:

$$z_{i,h}^{\text{HKS}} = \sum_{m=1}^K \exp(-\tau_h \lambda_m) \phi_m(i)^2. \quad (7)$$

这里, $z_{i,h}^{\text{HKS}}$ 表示节点 i 在第 h 个扩散尺度下的结构特征; τ_h 表示第 h 个扩散时间尺度; $\phi_m(i)$ 表示第 m 个特征向量在节点 i 上的值。

所以 Laplacian 降计算量的原因主要有三个:

1. 避免构造 **node-by-gene** 大表达块。
2. 利用稀疏图边, 而不是 **dense** 特征矩阵。
3. 把每个节点压缩到少量结构维度。

但要注意: Laplacian 只能降低特征构造和结构表征成本, 不能自动消除 **PIJ** 的 $n_s \times n_t$ 二次复杂度。这就是后面还要加 sparse OT、hierarchical OT、coarse-to-fine OT 的原因。

1.4 问题 4: JointNMF 不也是矩阵分解吗? 为什么要特殊处理? 为什么有泄露风险?

JointNMF 和 Laplacian 的最终作用相似: 都想把节点变成低维向量。但两者的输入和风险完全不同。

JointNMF 的基本形式是:

$$X_t \approx W_t H. \quad (8)$$

这里, $X_t \in \mathbb{R}_{\geq 0}^{n_t \times p}$ 表示时间点 t 的非负节点特征矩阵; $W_t \in \mathbb{R}_{\geq 0}^{n_t \times K}$ 表示节点的低维 NMF 表征; $H \in \mathbb{R}_{\geq 0}^{K \times p}$ 表示共享字典; n_t 表示时间点 t 的节点数量; p 表示输入特征维度; K 表示 NMF component 数。

它为什么要特殊处理? 因为 `clean_expression_cci_mix` 里的 X_t 仍然可能非常大:

$$X_t = [X_t^{\text{expr}}, X_t^{\text{CCI}}]. \quad (9)$$

这里, X_t^{expr} 表示表达基因块; X_t^{CCI} 表示 LR/CCI 特征块。NMF 虽然会把它分解成低维 W_t , 但在分解之前仍然要构造和反复乘这个大矩阵, 所以不一定省。

因此我建议把 JointNMF 的输入改成 compact CCI/DDI in-out 特征。对 LR pair q , 节点 i 的 compact 特征是:

$$X_{i,(q,\text{out})} = \sum_j \widetilde{M}_{ij}^{(q)}, \quad X_{i,(q,\text{in})} = \sum_j \widetilde{M}_{ji}^{(q)}. \quad (10)$$

这里, $\widetilde{M}_{ij}^{(q)}$ 表示归一化后的 LR pair q 从节点 i 到节点 j 的通信强度; $X_{i,(q,\text{out})}$ 表示节点 i 的发送强度; $X_{i,(q,\text{in})}$ 表示节点 i 的接收强度。

泄露风险来自两个地方:

1. **字典泄露**: 如果用所有时间点一起学习共享字典 H , 那么 target time 的统计结构也参与了 source embedding 的坐标系定义。做描述性分析时问题不大, 但如果我们把 PIJ 当成 $t_0 \rightarrow t_1$ 的 transition prediction, 就不够干净。

2. **特征筛选泄露**：如果 LR 子集 $\mathcal{Q}^*$ 是根据所有时间点，尤其 target time 的强度筛出来的，那么 source 节点特征已经偷偷看过 target。

更安全的版本是 source-only dictionary：

$$X_{t_0} \approx W_{t_0} H_{t_0}, \quad (11)$$

这里， X_{t_0} 表示 source time 的 compact 特征矩阵； W_{t_0} 表示 source 节点表征； H_{t_0} 表示只从 source time 学出来的字典。

然后固定 H_{t_0} ，把 target 投影进去：

$$W_{t_1}^* = \arg \min_{W \geq 0} \|X_{t_1} - W H_{t_0}\|_F^2 + \alpha \|W\|_F^2. \quad (12)$$

这里， $W_{t_1}^*$ 表示 target 节点在 source 字典下的表征； $\|\cdot\|_F$ 表示 Frobenius 范数； α 表示可选的稳定正则系数。

1.5 问题 5：普通 Laplacian eigenvector 跨图不可比是什么意思？

这个风险点很重要。Laplacian eigenvector 不是普通坐标轴，它有数学上的不唯一性。

第一种不唯一性是**符号翻转**：如果 ϕ 是特征向量，那么 $-\phi$ 也是同一个特征值的合法特征向量。这里， ϕ 表示某个 Laplacian 特征向量。

也就是说，同一个图结构，算法这次可能给出 ϕ ，下次可能给出 $-\phi$ 。如果我们直接把 $\phi(i)$ 当作节点坐标，那么同一个结构位置可能因为符号翻转而被误认为距离很远。

第二种不唯一性是**子空间旋转**：如果两个特征值非常接近，或者出现重根，那么对应的多个特征向量可以在同一个子空间里任意旋转。设两个合法基向量为 ϕ_1 和 ϕ_2 ，另一个合法表示可以是：

$$\begin{bmatrix} \psi_1 & \psi_2 \end{bmatrix} = \begin{bmatrix} \phi_1 & \phi_2 \end{bmatrix} R. \quad (13)$$

这里， ψ_1 和 ψ_2 表示旋转后的特征向量； R 表示一个二维正交旋转矩阵。

所以普通 Laplacian embedding 的风险是：不同时间点图的坐标轴可能不是同一套坐标轴。拿 t_0 的 eigenvector 坐标直接和 t_1 的 eigenvector 坐标算距离，可能把数值算法造成的符号/旋转差异误认为生物差异。

HKS 能缓解这个问题，因为它用的是：

$$z_{i,h}^{\text{HKS}} = \sum_{m=1}^K \exp(-\tau_h \lambda_m) \phi_m(i)^2. \quad (14)$$

平方项 $\phi_m(i)^2$ 消除了符号翻转；按特征值加权的求和更像节点在图上扩散后的局部结构指纹，而不是直接比较不稳定的坐标轴。因此，我建议普通 Laplacian embedding 只做 sanity check，主结果优先用 HKS 或 sign-anchored Laplacian。

2 现在的 OT 是否过于简单？

结论是：现有代码的 OT 求解器不算假 OT，但成本设计确实偏简单。

当前代码逻辑可以概括为两步。第一步，把多个成本分量做归一化加权平均：

$$C_{ij} = \frac{\sum_{r=1}^R \lambda_r \text{Normalize} \left(C_{ij}^{(r)} \right)}{\sum_{r=1}^R \lambda_r}. \quad (15)$$

这里, C_{ij} 表示 source 节点 i 到 target 节点 j 的总成本; $C_{ij}^{(r)}$ 表示第 r 个成本分量; λ_r 表示第 r 个成本分量权重; R 表示成本分量总数。

第二步, 把总成本送入熵正则 OT:

$$P^* = \arg \min_{P \in \mathcal{U}(a,b)} \sum_{i=1}^{n_s} \sum_{j=1}^{n_t} P_{ij} C_{ij} + \varepsilon \sum_{i=1}^{n_s} \sum_{j=1}^{n_t} P_{ij} (\log P_{ij} - 1). \quad (16)$$

这里, P^* 表示 OT coupling; $\mathcal{U}(a,b)$ 表示满足 source 边际 a 和 target 边际 b 的非负矩阵集合; a 表示 source mass 分布; b 表示 target mass 分布; ε 表示熵正则强度。

最后为了作为转移概率使用, 做行归一化:

$$\hat{P}_{ij} = \frac{P_{ij}^*}{\sum_{j'=1}^{n_t} P_{ij'}^*}. \quad (17)$$

这里, $\hat{P}_{ij}$ 表示 MIGNet 使用的 PIJ 条件转移概率。

所以问题不在于“没有 OT”, 而在于: 现在的 C_{ij} 太像人工拼出来的距离, 缺少图路径、层级结构、出生死亡、局部候选和跨时间动态约束。

3 更有创意且合理的 OT 方案

3.1 方案 A: Graph Geodesic OT, 用最短路径定义成本

你提到“比如最短路径”, 这个方向是合理的。但要注意: 最短路径本身不是 **PIJ**, 最短路径更适合作为 **OT** 的成本或 **prior**; **OT** 负责把这个成本变成满足全局质量约束的 **PIJ**。

构造一个跨时间候选图 $\mathcal{G}_{t_0, t_1}$ 。这里, $\mathcal{G}_{t_0, t_1}$ 表示把 source time 节点和 target time 节点放在一起的辅助图。图内可以包括三类边:

- source time 内部 CCI/DDI 边;
- target time 内部 CCI/DDI 边;
- source 到 target 的候选跨时间边, 例如 SR 接近、空间接近、结构 embedding 接近的 kNN 边。

定义节点 i 到节点 j 的图测地距离:

$$d_{\mathcal{G}}(i, j) = \min_{\pi: i \rightsquigarrow j} \sum_{e \in \pi} c_e. \quad (18)$$

这里, $d_{\mathcal{G}}(i, j)$ 表示辅助图上的最短路径距离; π 表示一条从 i 到 j 的路径; e 表示路径上的边; c_e 表示边 e 的代价。

然后用它作为 OT 成本:

$$C_{ij} = \lambda_{\text{geo}} \text{Normalize}(d_{\mathcal{G}}(i, j)) + \lambda_{\text{SR}} \text{Normalize}(|s_i - s_j|). \quad (19)$$

这里, λ_{geo} 表示图测地成本权重; λ_{SR} 表示 SR 成本权重; s_i 和 s_j 分别表示 source 节点 i 与 target 节点 j 的 SR 值。

这个方案比直接 cosine 距离更像“沿着 CCI/DDI 网络传播”。它适合作为第一批创新方法之一, 名字可以叫 `geodesic_sr_ot`。

3.2 方案 B: Fused Gromov-Wasserstein OT, 同时匹配节点特征和图内结构

普通 OT 只看 source 节点 i 和 target 节点 j 的一对一成本；它不显式要求“source 内部的邻近关系”在 target 中被保持。Gromov-Wasserstein 更适合比较两个图，因为它看的是图内距离结构是否一致。

设 $D_s(i, i')$ 表示 source 图中节点 i 和 i' 的图内距离， $D_t(j, j')$ 表示 target 图中节点 j 和 j' 的图内距离。这里， D_s 和 D_t 可以来自 shortest path、diffusion distance 或 resistance distance。

Fused Gromov-Wasserstein 的目标可以写成：

$$\min_{P \in \mathcal{U}(a, b)} (1 - \rho) \sum_{i, j} P_{ij} C_{ij}^{\text{feat}} + \rho \sum_{i, i', j, j'} (D_s(i, i') - D_t(j, j'))^2 P_{ij} P_{i'j'}. \quad (20)$$

这里， C_{ij}^{feat} 表示节点特征成本，例如 SR、HKS、compact NMF； $\rho \in [0, 1]$ 表示结构匹配项的权重； P_{ij} 表示 coupling 矩阵中 source 节点 i 到 target 节点 j 的质量。

这个方案最符合“网络结构真的参与 OT”的直觉。缺点是计算更重，所以不建议 spot 全量直接跑。更适合 domain 层，或者先用低秩/采样版本。方法名可以叫 fgw_sr_ot。

3.3 方案 C: Hierarchical Coarse-to-Fine OT, 先 domain 再 spot

为了解决 $n_s \times n_t$ 太大，最自然的方法是先在 coarse domain 层求一个上层 OT，再只在被上层认为可能对应的 domain pair 内细化到 spot 层。

设 source spot 集合为 V_s ，target spot 集合为 V_t ，source domain 集合为 U_s ，target domain 集合为 U_t 。这里， V_s 和 V_t 表示细粒度节点集合； U_s 和 U_t 表示粗粒度 domain 节点集合。

第一步，在 domain 层求：

$$P_U^* = \text{OT}(C_U, a_U, b_U). \quad (21)$$

这里， P_U^* 表示 domain-to-domain coupling； C_U 表示 domain 层成本； a_U 和 b_U 表示 domain 层边际分布。

第二步，只保留 top- k domain pair：

$$\Omega_U = \{(u, v) : v \in \text{TopK}(P_U^*(u, :))\}. \quad (22)$$

这里， Ω_U 表示允许细化的 domain pair 集合； u 表示 source domain； v 表示 target domain。

第三步，对每个 $(u, v) \in \Omega_U$ ，只在对应 spot 子集里求局部 OT：

$$P_V^{(u, v)} = \text{OT}(C_V^{(u, v)}, a_V^{(u)}, b_V^{(v)}). \quad (23)$$

这里， $P_V^{(u, v)}$ 表示 domain pair (u, v) 内部的 spot-to-spot coupling； $C_V^{(u, v)}$ 表示该局部块的 spot 成本； $a_V^{(u)}$ 和 $b_V^{(v)}$ 表示该局部块的边际分布。

这个方案既符合你的层级网络设定，也直接解决 spot 层 PIJ 太大的问题。它很适合作为主工程路线，名字可以叫 hierarchical_sr_ot 或 coarse_to_fine_ot。

3.4 方案 D: Sparse Candidate OT, 只在可信候选边上求 PIJ

dense OT 的问题是所有 i, j 都要算，即使很多 target 明显不可能对应。可以先构造候选集合 Ω ，然后约束非候选位置为 0。

定义候选集合：

$$\Omega = \{(i, j) : j \in \mathcal{N}_k(i) \text{ or } |s_i - s_j| \leq \delta_s \text{ or } \|x_i - x_j\| \leq \delta_x\}. \quad (24)$$

这里， Ω 表示允许传输的 source-target 对； $\mathcal{N}_k(i)$ 表示节点 i 的 target kNN； δ_s 表示 SR 差异阈值； δ_x 表示空间距离或结构距离阈值； x_i 表示节点 i 的空间坐标或结构 embedding。

稀疏 OT 约束为：

$$P_{ij} = 0, \quad (i, j) \notin \Omega. \quad (25)$$

这里， P_{ij} 表示 source 节点 i 到 target 节点 j 的传输质量。

如果每个 source 只保留 k 个候选 target，复杂度从 $O(n_s n_t)$ 近似降为：

$$O(k n_s). \quad (26)$$

这里， k 表示每个 source 节点保留的候选 target 数。

这个是最现实的工程优化。要注意保证每个 target 至少被若干 source 覆盖，否则 balanced OT 会不可行；可以加 target coverage repair。方法名可以叫 sparse_sr_ot。

3.5 方案 E: Unbalanced / Partial OT, 允许细胞状态出生和消失

发育数据里，source 和 target 节点不一定严格一一守恒。当前 balanced OT 的边际约束有时太硬，会强迫所有 source mass 都分给 target，也强迫所有 target mass 都被填满。

Unbalanced OT 可以写成：

$$\min_{P \geq 0} \sum_{i,j} P_{ij} C_{ij} + \varepsilon \sum_{i,j} P_{ij} (\log P_{ij} - 1) + \eta_s \text{KL}(P \mathbf{1} \| a) + \eta_t \text{KL}(P^\top \mathbf{1} \| b). \quad (27)$$

这里， η_s 表示 source 边际偏离惩罚； η_t 表示 target 边际偏离惩罚； $\text{KL}(\cdot \| \cdot)$ 表示 KL 散度； $P \mathbf{1}$ 表示 source 侧实际输出质量； $P^\top \mathbf{1}$ 表示 target 侧实际接收质量。

Partial OT 则只运输一部分总质量：

$$\sum_{i,j} P_{ij} = m, \quad (28)$$

这里， m 表示需要运输的总质量，通常 $0 < m \leq 1$ 。

这个方案解决的是生物合理性：发育过程中状态会分化、消失、增殖，不一定所有节点都应强制匹配。当前代码已经有 unbalanced 参数基础，可以优先利用。

3.6 方案 F: Diffusion Prior / Schrödinger Bridge, 把网络扩散作为先验

可以先从 CCI/DDI 图上构造一个扩散先验 R_{ij} ，再用 OT 修正它。直觉是：如果 CCI/DDI 网络本身暗示 i 更可能到 j ，那么 P_{ij} 不应该只由几何距离决定。

定义先验 kernel：

$$R_{ij} = \exp \left(-\frac{d_G(i, j)}{\sigma} \right). \quad (29)$$

这里， R_{ij} 表示从 source 节点 i 到 target 节点 j 的图扩散先验； $d_G(i, j)$ 表示跨时间辅助图上的最短路径距离； σ 表示扩散尺度。

再定义带先验的 kernel:

$$K_{ij} = R_{ij} \exp\left(-\frac{C_{ij}}{\varepsilon}\right). \quad (30)$$

这里, K_{ij} 表示 Sinkhorn 使用的初始 kernel; C_{ij} 表示节点成本; ε 表示熵正则强度。

这个方案比“直接距离”更像生物网络上的传播过程。它可以和 shortest path、Laplacian-HKS、SR 一起用。方法名可以叫 diffusion_prior_ot。

4 怎么解决 PIJ 仍然是二次复杂度的问题?

路线	复杂度改善	建议优先级
Sparse Candidate OT	从 $O(n_s n_t)$ 降到近似 $O(k n_s)$ 。	第一优先级。spot 层必须考虑。
Hierarchical Coarse-to-Fine OT	先 domain 后 spot, 只细化高概率 domain pair。	第一优先级。最符合你的层级模型。
GPU float32 Sinkhorn	不改复杂度, 但显存从 float64 降到 float32, 速度可能更好。	工程优先级高, 但不是根本解决。
Blockwise / streaming cost	不一次性保留所有成本矩阵, 分块计算。	用于避免 OOM, 但 Sinkhorn 仍需要特殊实现。
Low-rank / Nyström Sinkhorn	用低秩 kernel 近似避免 dense $n_s n_t$ 。	中期可做, 工程复杂度较高。
Mini-batch OT	每次只采样部分 source/target。	可做快速探索, 但指标稳定性要验证。
FGW on domain only	避免 spot 全量 FGW。	适合作为结构主结果, 不适合 spot 全量。

建议第一版不是直接上最复杂的 FGW, 而是按下面顺序:

1. laplacian_sr_ot: 先验证结构低维特征 + SR 是否有效。
2. sparse_laplacian_sr_ot: 在第一版上加 kNN 候选, 解决 spot 层 $n_s n_t$ 过大。
3. hierarchical_sr_ot: 先 domain-level PIJ, 再局部 refine spot-level PIJ。
4. geodesic_sr_ot: 把最短路径作为成本, 增强“沿网络传播”的解释性。
5. fgw_sr_ot: 作为更强但更重的结构 OT, 对 domain 层做主结果或补充验证。

5 更新后的实施建议

5.1 新增 profile-only 工具

建议新增 scripts/profile_vertical_runtime.py, 输出一个 CSV:

- organ、time、lower layer、upper layer;
- lower/upper 节点数;
- lower/upper 边数;
- LR 文件数;
- 每个 PIJ kernel 的 $n_s \times n_t$;
- CPU/GPU 估计内存;
- 按 pilot runtime 外推的预计耗时。

5.2 现有代码最小改动

1. 给 `run_mignet_vertical_ablation.py` 补上 `--progress`、`--progress-log`、`--pij-device` 等参数, 和 `run_mignet_vertical.py` 对齐。
2. 新增 `clean_cci_ddi_graph`, 参照 `clean_expression_cci_mix` 的路径、CCI 建图、native units、overlap 输出, 但不构造表达大矩阵。
3. 新增 `laplacian_sr_ot`, 优先 HKS; 普通 eigenvector 只做 sanity check。
4. 新增 `joint_nmf_sr_ot`, 输入 compact CCI/DDI in-out 特征, 不吃 expression 大矩阵。
5. 新增 `sparse_ot` 支持: 允许 `component_builder` 返回 candidate mask 或 sparse cost support。
6. 新增 `hierarchical_sr_ot`: 先 upper/domain, 再 lower/spot block refine。

5.3 第一轮实验矩阵

项目	建议设置
network	<code>clean_cci_ddi_graph</code>
pilot	heart, 11.5 到 12.5, 一个 adjacent pair
第一批 PIJ	<code>laplacian_sr_ot</code> , <code>sparse_laplacian_sr_ot</code> , <code>hierarchical_sr_ot</code>
第二批 PIJ	<code>geodesic_sr_ot</code> , <code>joint_nmf_sr_ot</code>
补充结构法	<code>fgw_sr_ot</code> , 先只在 domain 层跑
输出	metrics + full sparse PIJ + units.csv + runtime profile, 不导出 raw native features

6 最终结论

1. 你完全可以先用输入大小估算 runtime; 当前代码已经有 PIJ estimated bytes 的基础, 但建议补 profile-only 脚本和 ablation progress 参数。
2. 报告中的 Laplacian/JointNMF 不是简单复用现有方法, 而是要新增 `laplacian_sr_ot` 和 `joint_nmf_sr_ot`。
3. Laplacian 的确是图结构降维, 能绕开表达大矩阵, 但不能单独解决 PIJ 二次复杂度。

4. JointNMF 也是降维, 但它依赖输入矩阵和共享字典; 如果输入仍是大矩阵就不省, 如果字典/特征筛选用了 `target time` 就有泄露风险。
5. 普通 `eigenvector` 跨图不可比是符号翻转和子空间旋转问题; HKS 或 `sign anchoring` 是为了让结构特征更稳定。
6. 当前 OT 不是假 OT, Sinkhorn 求解是真的; 但成本设计太粗, 应该升级为 `geodesic OT`、`sparse OT`、`hierarchical OT`、`unbalanced/partial OT`、`diffusion-prior OT`、FGW 等更有结构含义的方法。

最推荐的工程路线: 先做 `profile-only`, 再做 `laplacian_sr_ot`, 然后立刻加 `sparse_laplacian_sr_ot` 或 `hierarchical_sr_ot`, 否则 `spot` 层的 $n_s \times n_t$ 仍然会成为瓶颈。